Storage & City Watch — LA Council Tracker

Storage & City Watch — decisions, measurements, and where things stand

LA Council Tracker · 7 August 2026

Every number here was measured on the actual corpus unless explicitly labelled an estimate.

---

1. The decision that changed everything

Two design documents shaped this build:

- **City Watch.md** — a lessons-learned report from a prior agent running a municipal tracker. Core thesis: *a municipal watch tool fails silently by default*. "Nothing happened" and "the pipeline is broken" produce identical output unless you design against it.
- **City Document Parsing Schema — Design.md** (RIP Los Angeles) — whose central insight is **treat the PDF as disposable**: keep `full_text` + `source_url` + `sha256` forever, delete the file, re-fetch if ever needed.

That second idea turned out to be worth roughly two orders of magnitude here.

The measurement

Document PDFs	**637 MB**
The same content as extracted text	**10.8 MB**
Ratio	**59× — 98.3% eliminated**

This is not lossy. Every deleted PDF has a recorded `sha256` and a live `source_url`. A 2013-era City Clerk `onlinedocs` URL was tested and still returns **HTTP 200** — those paths are stable. The deletion is reversible; the originals sit on the City's servers.

The sweeper (`doc_retention.py`) has three safety properties: **dry-run is the default** (`--delete` required), it **refuses to delete on a sha256 mismatch** rather than destroying evidence of a problem, and it **never touches** anything whose `source_url` isn't a real re-fetchable URL or that's flagged `never_delete`.

Running total

	Before	Now
`documents/`	649 MB	17 MB
`corpus.db`	—	~59 MB
`transcripts/` (raw JSON, now duplicated in DB)	44 MB	44 MB
Total	**693 MB**	**~120 MB**

5.8× smaller, with more searchable content than before.

---

2. Do we need the 5 TB drive?

For this project: no. For the Morris Kight archive: yes, that's 118 GB of video and audio and it's the real consumer.

Projections from measured rates — presented as ranges because document counts per file vary enormously (one oil-drilling ordinance is 73 documents and 94% of the current corpus):

Scale	Text storage
1,000 council files	~0.5–2 GB
10,000 council files	~5–20 GB
25 years of every meeting transcript	**~0.5 GB**

Even at the pessimistic end that's **20–25 GB — about half a percent of a 5 TB drive.**

The irony worth recording: **dropping the PDFs is what made the drive unnecessary.** Had we kept them, 10,000 council files would have been ~96 GB and growing, and the drive would have been a hard requirement rather than a convenience.

---

3. LAN storage — yes for files, no for the database

This is not a preference. It is a correctness constraint.

Why the database can't live on a network share

- SQLite's locking depends on **POSIX advisory locks**, which are unreliable over SMB and NFS. Concurrent access can corrupt the file.
- This schema uses **WAL mode**, which **does not work over a network filesystem at all** — it requires shared memory that doesn't cross a network boundary.

The latency question, measured and reasoned

Local baseline, median of 8 runs on the real corpus:

Operation	Local
FTS phrase search (transcripts)	**0.1 ms**
FTS search (unified index)	**0.0 ms**
Decompress one meeting's segments	**3.7 ms**

Those are effectively instant. Over a LAN they would not be, and the reason is structural rather than bandwidth-related: **SQLite reads in 4 KB pages, and a single query may touch hundreds of them at random offsets.** Bandwidth is irrelevant to that pattern; per-operation round-trip time is what dominates.

Estimate, not measurement (no share was mounted to test): on wired gigabit with ~0.2–1 ms round trips, a query touching a few hundred uncached pages lands in the tens-to-hundreds of milliseconds — call it **100–1000× slower** than the numbers above. SMB client caching softens this in practice, but it is the wrong shape of workload for a network filesystem regardless.

Sequential bulk reads are a different story. Pulling a whole PDF or a transcript JSON is bandwidth-bound, not latency-bound, and a LAN handles that fine.

Recommended split

```
LOCAL (internal or directly-attached Thunderbolt)
    corpus.db          <- 59 MB. Small enough to always live local.

LAN / NAS - fine
    bulk archival files, backups, video, the Morris Kight material
```

Your PDF decision is what makes this clean: at 59 MB the database never needs to be anywhere but local.

---

4. The storage switch

`scripts/storage.py` is the single place data location is decided. Default is internal — no configuration needed, nothing breaks.

```
// config.json
"data_root": "/Volumes/Archive/la-council-tracker"   // external
"data_root": null                                     // internal (default)
```

Or per-run: `LA_TRACKER_DATA_ROOT=/Volumes/... python3 check_updates.py`

Two guards, both because the failure modes are silent:

Mount guard. If `data_root` is set but the volume isn't mounted, `os.makedirs()` would happily rebuild the tree on the internal disk — leaving two partial archives, a health check reporting green, and cumulative silent corruption. So a missing sentinel file (`.tracker_volume_id`) means the script **refuses to run** rather than guessing.

Network-filesystem guard. `db_path()` detects SMB/NFS/AFP mounts and **refuses to put `corpus.db` there**, for the locking and WAL reasons above.

Check current state any time: `python3 scripts/storage.py`

5. City Watch schema — what's built

Piece	Status
SQLite + FTS5 corpus (`db.py`)	**Built** — documents, public comments, transcripts, emails, outbox, unified search index
Corpus migration (`migrate_to_db.py`)	**Built** — 134 docs, 344 comments, 219 meetings
Public-comment parsing	**Built** — see below
Retention sweeper (`doc_retention.py`)	**Built** — 663 MB deleted, dry-run default
Search tool (`corpus_search.py`)	**Built** — *"build the reader before adding more writers"*
Transcript storage optimisation	**Built** — 207 MB → 58 MB
Storage switch (`storage.py`)	**Built** — internal default, external opt-in
Document backfill (`backfill_documents.py`)	**Running**
Email ingestion (`email_ingest.py`)	**Built**, awaiting subscriptions
Send semantics (`send_digest.py`)	**Built and tested**, awaiting SMTP
Stage model + upcoming-agenda scanner	**Not built**

Public comment — the 8× finding

The archived PDFs bundle many comments each. 42 PDFs contained **328 individual comments from 315 distinct people**, but the old tooling printed one 300-character excerpt per *file* — leaving roughly **87% invisible**. Parsing to one row per comment made it all searchable and enabled per-commenter aggregation, which immediately surfaced things like an oil company filing six comments across two name variants.

93% of comments are structured web-form submissions; the remaining ~7% are attached letters (often the most substantive) and are flagged `needs_review` rather than force-parsed into a fabricated commenter.

Transcript coverage — the other big gap

The earlier keyword searches filtered on `title == 'City Council Meeting'`, which silently excluded **367 committee meetings** — including 12 Housing and Homelessness Committee sessions directly relevant to the investigation. Committee is typically where the substantive argument happens; full Council often adopts committee reports with little debate.

Now fetching all meetings with video, SAP-deduplicated (the – SAP suffix is the Spanish-language audio track — same proceeding, ~half of all records). 2026: **226 unique meetings, 203 fetched, zero failures, 44 MB.**

Send semantics — the "unfixable later" item

City Watch.md is blunt that this must be right on day one. Properties **tested, not assumed**:

- **Bare invocation printed the digest and sent nothing.** `--send` is required.
- **`--send` with no credentials refused and left both test items queued** — the seen-pointer does not advance on failure, so nothing is silently dropped.
- `sent_digests` records *which items* went out, so "was this in Tuesday's digest?" is answerable from the log.
- The digest states its own coverage, so silence is attributable.
- A stale last-sent date is **not** a fault — quiet weeks are legitimate.

---

6. Bugs found and fixed along the way

- **Transcript→agenda-item attribution was confidently wrong.** Video anchors are coarse (one meeting: 54 agenda items sharing 20 anchors), so remarks during general public comment were pinned to unrelated items. Verified case: a comment about **LAFLA legal-aid contracts** was attributed to an **oil-and-gas drilling ordinance**, which was then auto-added to the watchlist. Fixed by removing auto-attribution entirely — hits are surfaced with timestamped video links for human judgement.
- **Staleness indicator was lying.** The digest reported "latest May 29, 2026" while August meetings existed: `MAX()` on display-format dates sorts `"May"` above `"Aug"` as a string. Added an ISO date column, backfilled 219 rows. This one mattered disproportionately — a false staleness signal is exactly the failure City Watch.md is about.
- **61 of 66 watched files had no documents archived.** They were baselined before document-tracking existed, so the tracker considered their documents already "seen"

and would never have gone back. Backfill running now.
- **Wasted indexes.** A 30 MB index on a ~40-value column across 611k rows; a 36 MB UNIQUE index covering an entire text column; 35 MB of FTS duplicating content it had been handed.

---

7. What's next

1. **Finish the backfill** (running).
2. **Stage model + upcoming-agenda scanner.** The tracker is currently entirely backward-looking — it reads City Clerk's activity log, which is a record of the past. `primegov_client.list_upcoming_meetings()` already works and nothing reads it. Wiring it in gives pre-vote alerts: *"CF 24-0909 is on the Council agenda in 5 days."* Highest value per unit of work, and it needs no credentials.
3. **Email subscriptions** — ENS and per-council-file. Note these aren't one firehose: City Clerk is one signup *per file*, ENS is a separate area/topic signup. Gmail with 2FA requires an **App Password**, not the account password.
4. **Prune the watchlist** — 30 of 66 files are already past expiration.
5. **Move to external storage** when the drive arrives — one config line, plus a verified copy.

A note on lead time

Agendas post close to California's Brown Act floor — **72 hours** for regular meetings, 24 for special ones. That makes a **daily** check the minimum viable cadence: a check run "when you think of it" will routinely miss the window entirely. It also means scheduling and delivery have to land *with* the scanner, not after it — a scanner nobody is reading is a dead feature.